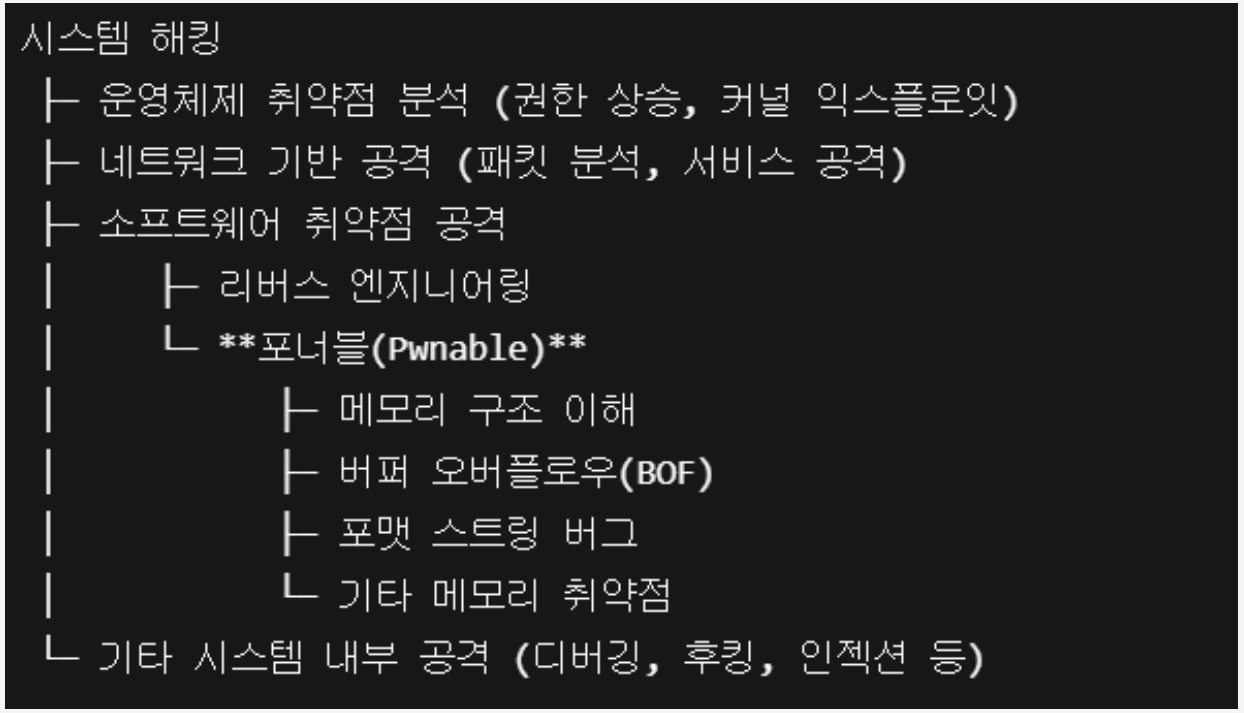
시스템 해킹(pwnable)

포너블(pwnable) 개요

시스템 해킹이란 무엇인가...

시스템 해킹이라며...

- 시스템 해킹 : 프로그램이나 시스템의 취약점을 분석하고, 이를 이용해 권한을 상승시키거나 임의의 코드를 실행하는 일련의 과정



너무 넓다!

→ 포너블에 집중

◀ 지피티 피셜 시스템 해킹 분야

포너블이란?

- CTF의 문제 유형
- “시스템 해킹 분야의 한 갈래로, 프로그램의 메모리 취약점을 이용해 시스템을 장악하거나 동작을 조작하는 기술”
- 주로 운영체제나 프로그램의 보안 취약점을 분석하고 공격하여, 관리자 권한을 획득하거나 프로그램의 흐름을 해커가 원하는 방식으로 바꾸는 것이 목적

어디에 쓰나요?

- 해킹 연습이 된다.
- 포너블 공부한 사람도 취약점 제보를 할 수 있다.
  - nginx, pppd, sudo 등 여러 어플리케이션들이 C로 작성되어있음.
  - 포너블로 공부한 취약점이나 공격기법이 실제 서비스에도 적용될 수 있음.
  - 예시) CVE-2020-8597 : pppd의 EAP 패킷 처리 중 BOF 취약점 발견. 임의 코드 실행이 가능해 9.8 CRITICAL을 받음!
- CTF에서 좋은 스킬셋이 되어준다.

세션소개

우리 뭐하나요

세션소개

- 소개 : EVI$ION의 YB들을 위한 시스템 해킹 입문 강의
- 목표 : 포너블 입문하기! Buffer Overflow 취약점을 이해하면 성공

커리큘럼

- 1회차: 메모리 구조 & 스택 프레임
  - 컴퓨터가 프로그램을 실행할 때, 메모리를 어떻게 쓰는지 알아보고
  - gdb로 실제 실행 중인 프로그램 직접 분석해보기
- 2회차: BOF & ret2win
  - 해킹해서 프로그램 실행 흐름 바꿔보기

진행 방식

- 이론 + 실습 + 과제
- 복습 목적 과제가 나가요
  - 과제에는 AI/인터넷 검색 허용
  - 제출은 다음 세션 시작 전(18:30) 까지, 늦제출은 금요일(23:59)까지
- 어려워요 → 저도요
  - 질문해주세요.
  - “이거 너무 기초적인 질문인데…” → 대환영

메모리 구조 & 스택 프레임

목차

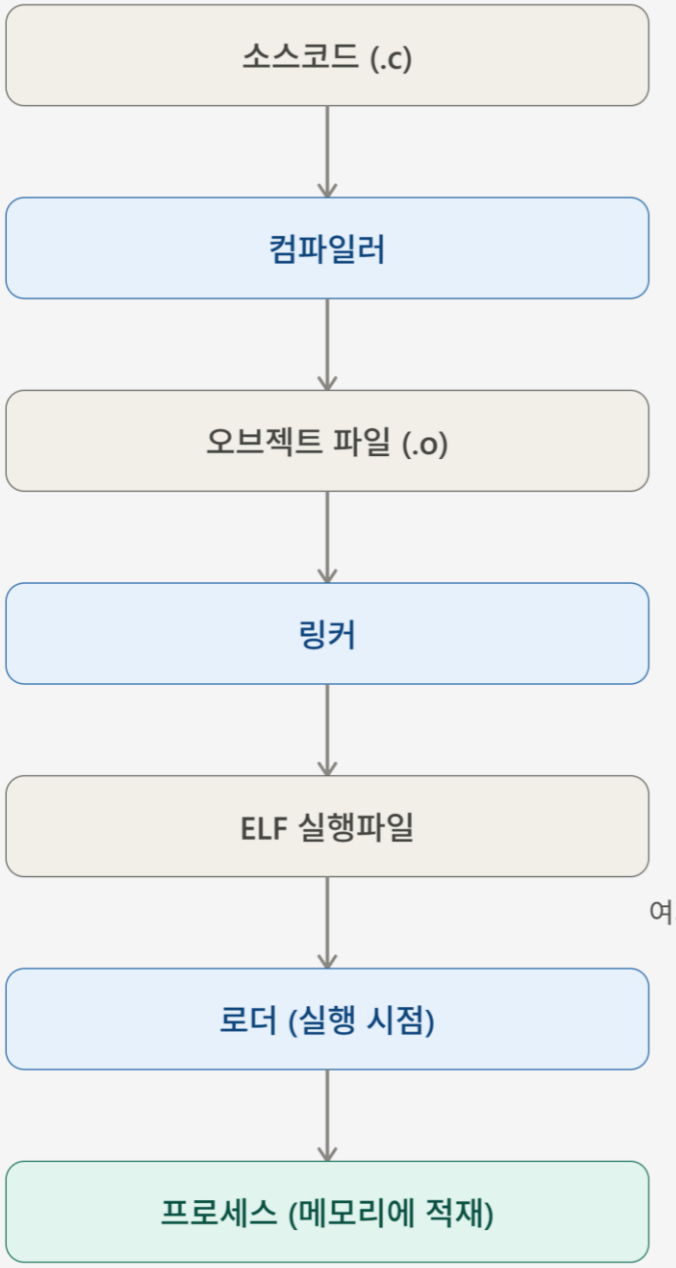
- 메모리 구조
- 레지스터
- 스택 프레임
  - 함수호출규약
  - 프롤로그/에필로그
- 실습: GDB로 프로그램이 실행되는 거 보기

메모리 구조

프로그램이 메모리를 어떻게 사용하는가

프로세스

- 소스코드: 작성한 코드파일
- 프로그램: 빌드한 실행파일
- 프로세스: 프로그램 실행한 거



프로세스

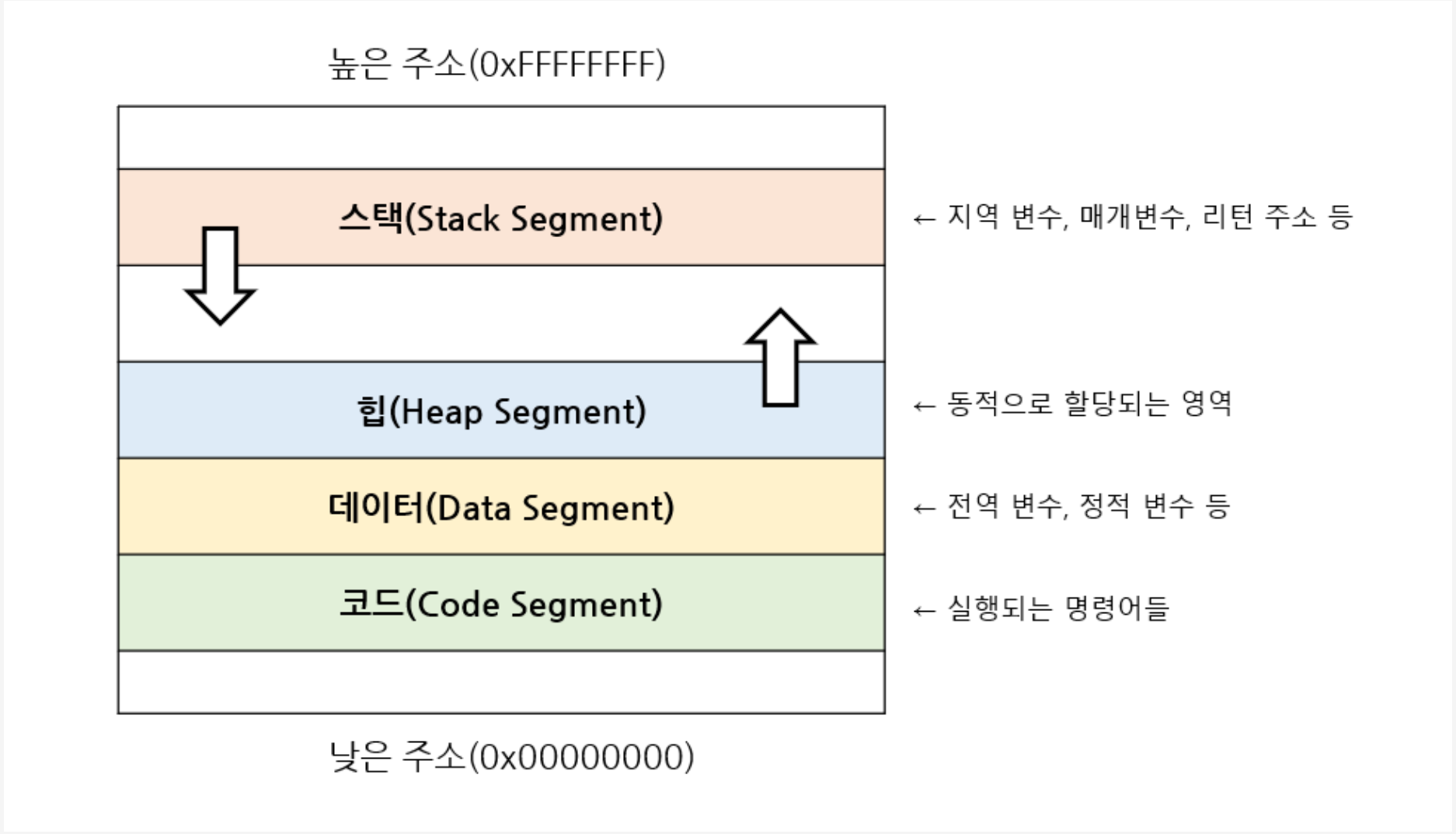
```
tester@LAPTOP-O928K059:~$ vim meow.c
tester@LAPTOP-O928K059:~$ cat meow.c
#include <stdio.h>

int main(){
    puts("meow ^._.^");
    return 0;
}
tester@LAPTOP-O928K059:~$ gcc -o meow meow.c
tester@LAPTOP-O928K059:~$ ls -l
total 20
-rwxrwxr-x 1 tester tester 15960 Jul 23 21:00 meow
-rw-rw-r-- 1 tester tester    72 Jul 23  2026 meow.c
tester@LAPTOP-O928K059:~$ ./meow
meow ^._.^
tester@LAPTOP-O928K059:~$
```

메모리 구조: 4영역

- 코드 : 실행 코드가 들어있는 공간
- 데이터 : 초기화된 데이터가 저장되는 공간
- 힙 : 프로그램 실행 중 동적으로 사용되는 공간
- 스택 : 함수 호출 시 인자, 지역 변수, 리턴 주소 등이 저장되는 공간

프로그램 메모리 구조 4영역



코드(Code)

- 코드 세그먼트(Code Segment) / 텍스트 세그먼트(Text Segment)
- 시스템이 알아들을 수 있는 실행 명령어(기계어)가 저장되어 있다
- 컴파일된 함수 코드와 제어 흐름 관련 명령어들 포함
- 실행과 읽기 권한만 부여

데이터(Data)

- 프로그램 실행 시 사용되는 데이터가 저장됨.
- 전역 변수(global variable)와 정적 변수(static variable)들이 들어감.
- 세분화한 구조
  - 데이터 세그먼트(.data) : 초기화 O 전역 변수, 정적 변수.
  - BSS 세그먼트(.bss) : 초기화 X 전역 변수, 정적 변수.
  - 리소스 데이터 세그먼트(.rodata) : 초기화 O 읽기 전용. (주로 상수)

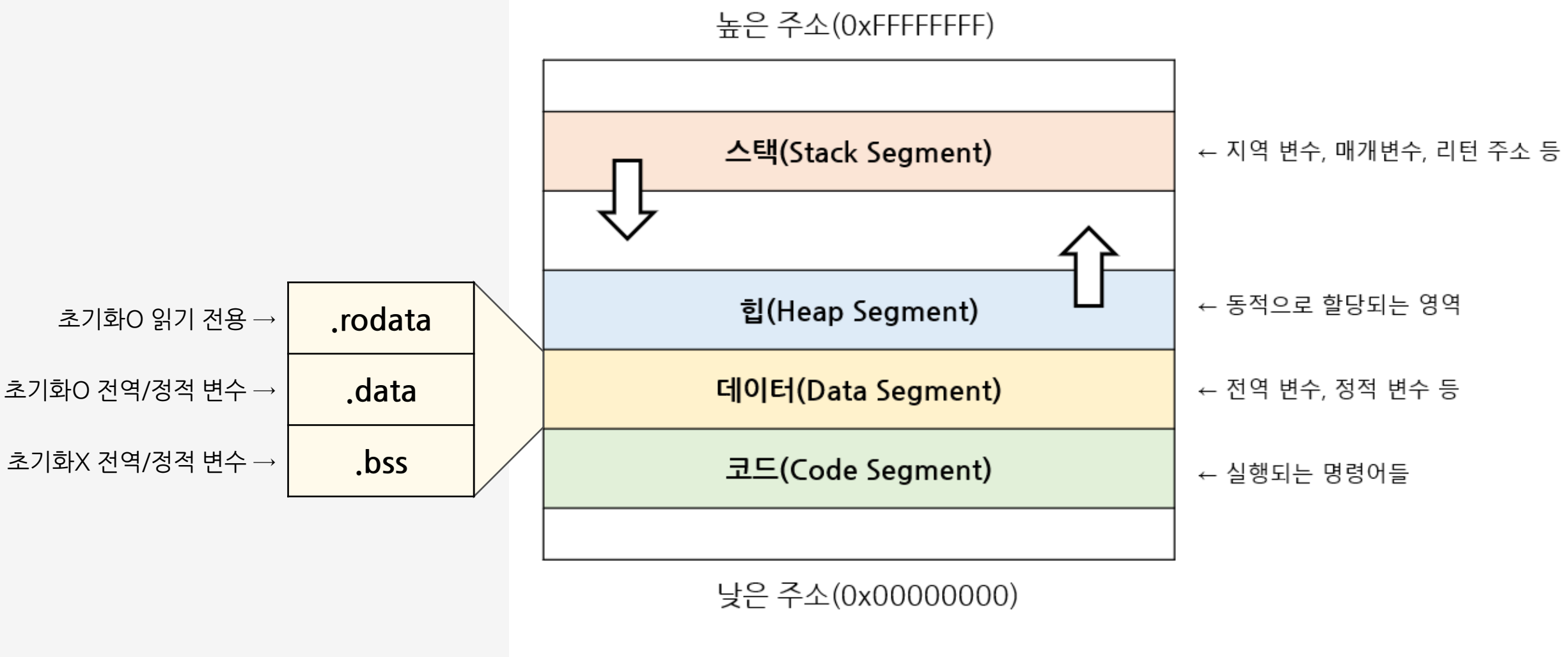
힙(Heap)

- 프로그램이 실행 중에 동적으로 사용되는 영역
- 하위 메모리 주소에서 상위 메모리 주소로 확장
- malloc, calloc, new 등에 의해 실행 중 동적으로 할당
- 개발자가 직접 free() 또는 delete()로 해제해야 함
- 메모리를 해제하지 않고 반복적인 메모리 할당이 누적되면 메모리 누수 (Memory Leak) 발생

스택(Stack)

- 프로그램 로직이 동작하기 위한 인자와 프로세스 상태가 저장되는 영역
- 함수 호출 시 지역 변수, 매개변수, 반환 주소 등이 저장
- 상위 메모리 주소에서 하위 메모리 주소로 확장

프로그램 메모리 구조 4영역



레지스터

CPU가 쓰는 아주 작은 저장공간

레지스터란?

- CPU 내부의 연산용 임시 저장소
- CPU 연산과 어셈블리어의 동작에 사용된다
- 범용 레지스터, 세그먼트 레지스터, 포인터 레지스터, 인덱스 레지스터, 플래그 레지스터 등으로 구분
- 각각의 레지스터는 특정 연산이나 메모리 접근에 특화

레지스터 종류

- 범용 레지스터(General-Purpose register)
  - 데이터 저장, 연산, 주소 계산 등 다목적으로 사용되는 레지스터
- 세그먼트 레지스터(Segment register)
  - 코드 세그먼트, 데이터 세그먼트, 스택 세그먼트를 가리키는 주소(base 주소)가 들어 있는 레지스터
- 포인터 레지스터(Pointer register)
  - 원래 범용 레지스터에 속하지만, 스택 프레임, 스택 최상단, 명령어 포인터 등 주소를 가리키는 특별한 용도로 자주 쓰임

레지스터 종류

- 인덱스 레지스터(Index register)
  - 배열, 문자열 처리 등에서 인덱스로 사용하는 레지스터
- 플래그 레지스터(Flag register)
  - 연산 결과 및 시스템 상태와 관련된 여러 가지 플래그 값들이 저장됨

[참고] 레지스터 종류

범용 레지스터

레지스터	비트	설명
EAX (Accumulator)	32	연산 결과, 함수 리턴 값 저장.
EBX (Base)	32	특정 주소 저장. 주소 계산을 위해 사용.
ECX (Counter)	32	루프 반복 횟수나 시프트 비트 수 기억.
EDX (Data)	32	일반 데이터 저장. 곱셈/나눗셈 시 보조.

세그먼트 레지스터

레지스터	비트	설명
CS (Code Segment)	16	실행할 코드가 있는 세그먼트.
DS (Data Segment)	16	일반 데이터가 있는 세그먼트.
SS (Stack Segment)	16	스택 영역의 베이스 주소.
ES, FS, GS	16	여분의 레지스터.

인덱스 레지스터

레지스터	비트	설명
EDI (Destination Index)	32	목적지 인덱스. 목적지 주소의 값 저장.
ESI (Source Index)	32	출발지 인덱스. 출발지 주소의 값.

포인터 레지스터

레지스터	비트	설명
EBP (Base Pointer)	32	베이스 포인터. 함수 호출 시 스택 프레임의 기준점.
ESP (Stack Pointer)	32	스택 포인터. 스택 최상단(top)을 가리킴.
EIP (Instruction Pointer)	32	명령 포인터. 다음에 실행할 명령어의 주소.

플래그 레지스터

분류	레지스터	설명
상태 플래그	CF (Carry Flag)	연산 수행에서 carry 혹은 borrow 발생.
	PF (Parity Flag)	최하위 비트가 0인 경우.
	AF (Adjust Flag)	산술 연산에서 BCD 숫자 캐리 발생.
	ZF (Zero Flag)	연산 결과가 0인 경우.
	SF (Sign Flag)	연산 결과가 음수인 경우.
	OF (Overflow Flag)	산술 연산 결과가 표현 가능 범위 초과.
제어 플래그	IF (Interrupt Enable Flag)	인터럽트가 사용가능한 경우
	DF (Direction Flad)	설정 시 문자열 연산에서 역순으로 읽음.
시스템 플래그	VM (Virtual 8086 Mode)	Virtual 8086 모드 활성화 여부.
	RF (Resume Flag)	디버깅 시 예외 재발 방지.
	AC (Alignment Check)	메모리 정렬 검사

이 친구들을 알아야 해요

- 스택 포인터 : ESP/RSP – 스택의 최상단
- 베이스 포인터 : EBP/RBP – 함수의 기준점
- 명령어 포인터 : EIP/RIP – 지금 실행 중인 명령어 위치
- 연산 결과용(관습적) : EAX/RAX – 연산 결과/리턴값



* R- : 64bit 아키텍처
* E- : 32bit 아키텍처

스택 프레임

프로그램을 실행할 때 스택을 어떻게 써먹을 것인가

함수호출규약

- 함수 호출 시 인자 전달, 리턴 값 처리, 스택 정리 방식을 정의하는 규칙
- 호출자(caller)와 callee 사이의 약속
- 아키텍처마다 다름!
- x86-32bit architecture cdecl calling convention
  - 인자를 오른쪽에서 왼쪽으로 스택에 push : func(a, b)면 b, a 순서로 넣음
  - 리턴 값은 EAX 레지스터에 저장됨 : return 5; 하면 EAX에 5 저장
  - 인자는 호출자(caller)가 정리 : 인자 넣은 쪽에서 인자 정리도 하기

프롤로그/에필로그

- 함수 진입과 종료 시 스택 프레임을 설정하고 복원하는 코드
  - 프롤로그: 함수 시작 시 스택 프레임을 설정하고 함수 실행에 필요한 환경을 구성하는 과정.
  - 에필로그: 함수 종료 시 스택 프레임을 정리하고 이전 실행 환경을 복원하는 과정
- 아키텍처 마다 다르지만 목적은 동일

함수 시작 : 프롤로그(Prologue)

```
push ebp              ; 이전 ebp 백업
mov ebp, esp          ; ebp를 현재 스택프레임 기준점으로
sub esp, 0x10         ; 로컬 변수 공간 확보
```

- 목적 : 새로운 스택 프레임 만들기

1. 이전 ebp 백업
2. ebp를 현재 스택프레임 기준점으로 변경
3. 로컬 변수 공간 확보

함수 시작 : 프롤로그(Prologue)

caller

```
push argv
call function
    : push retrun address
      callee로 이동
```

callee

```
push ebp
mov ebp, esp
sub esp, 0x10
```



함수 시작 : 프롤로그(Prologue)

caller

```
push argv
call function
    : push retrun address
      callee로 이동
```

callee

```
push ebp
mov ebp, esp
sub esp, 0x10
```



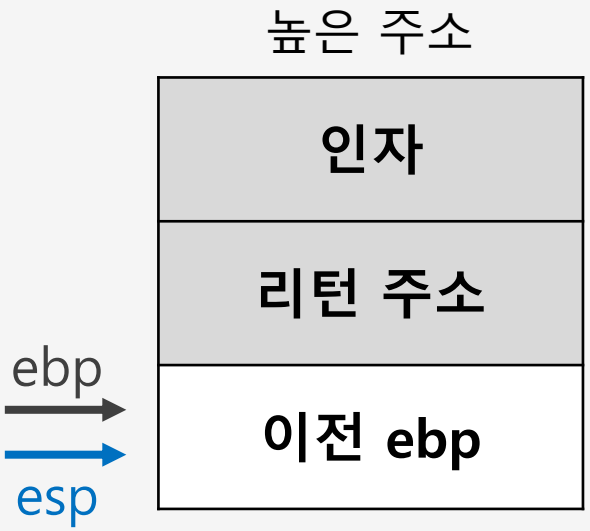
함수 시작 : 프롤로그(Prologue)

caller

```
push argv
call function
    : push retrun address
      callee로 이동
```

callee

```
push ebp
mov ebp, esp
sub esp, 0x10
```



함수 시작 : 프롤로그(Prologue)

caller

```
push argv
call function
    : push retrun address
      callee로 이동
```

callee

```
push ebp
mov ebp, esp
sub esp, 0x10
```



함수 종료 : 에필로그(Epilogue)

```
leave           ; mov esp, ebp + pop ebp
ret             ; pop eip, jmp eip
```

- 목적 : 스택 프레임 정리하기

1. (leave) mov esp, ebp : 로컬 변수 공간 제거
2. (leave) pop ebp : 이전 ebp로 ebp 복원 (caller의 ebp 복원)
3. (ret) pop eip : 스택에서 리턴 주소를 꺼내 eip에 넣기
4. (ret) jmp eip : 리턴 주소로 복귀 (caller로 점프)

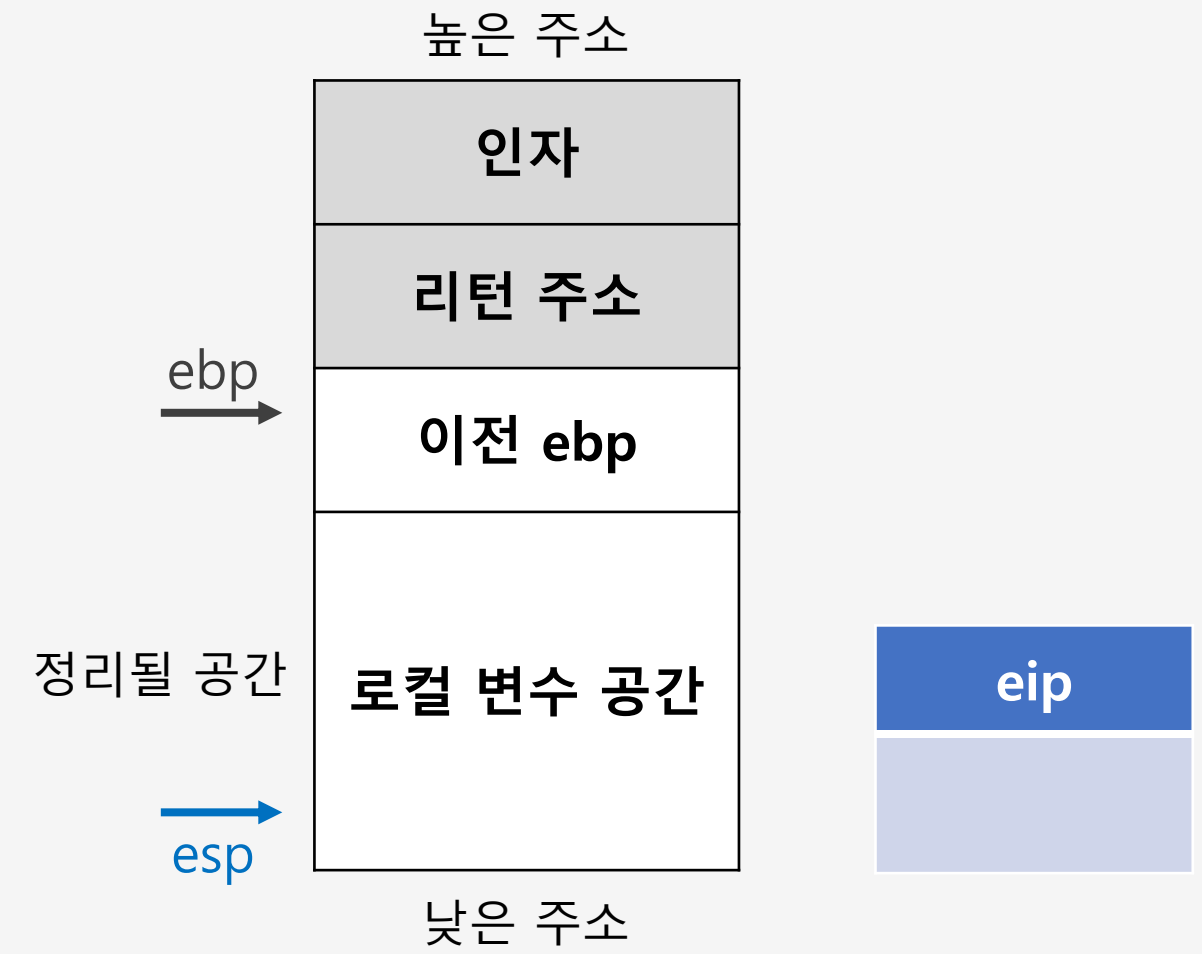
함수 종료 : 에필로그(Epilogue)

callee

```
leave
    : mov esp, ebp
      pop ebp
ret
    : pop eip
      jmp eip
```

caller

```
add esp, 0x4
```



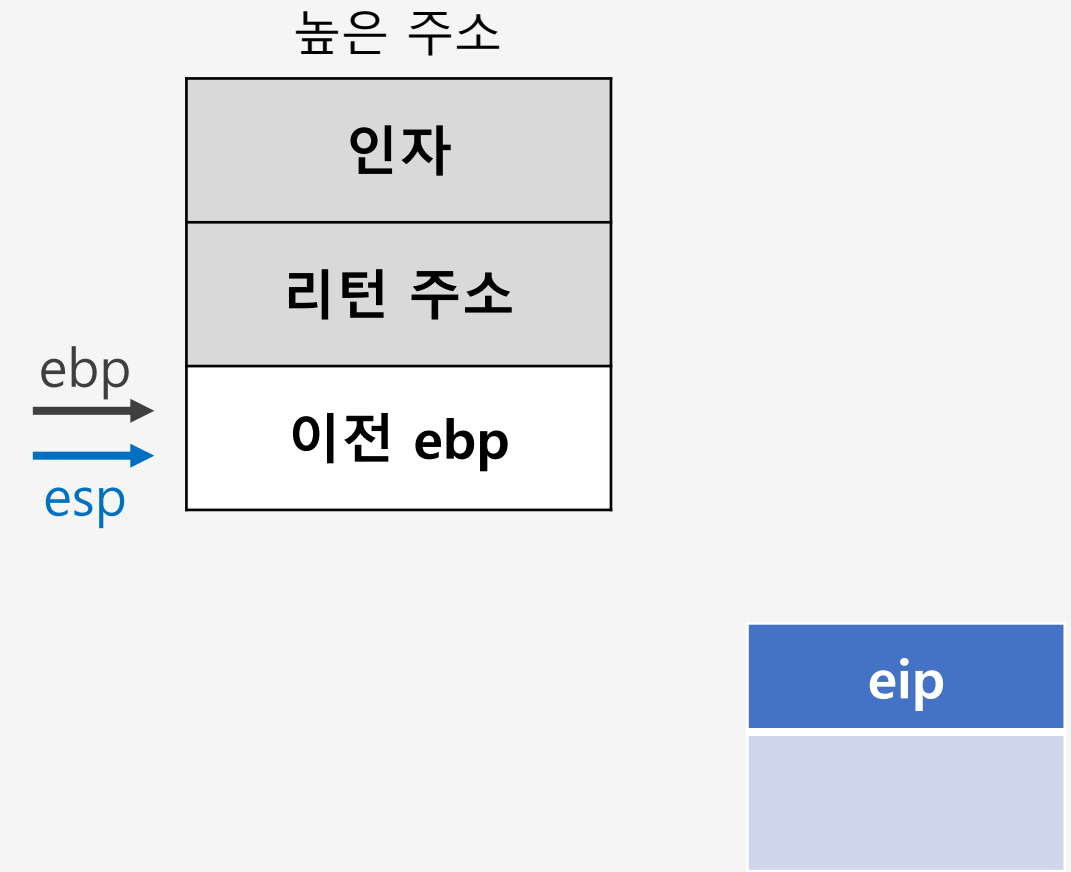
함수 종료 : 에필로그(Epilogue)

callee

```
leave
    : mov esp, ebp
      pop ebp
ret
   : pop eip
     jmp eip
```

caller

```
add esp, 0x4
```



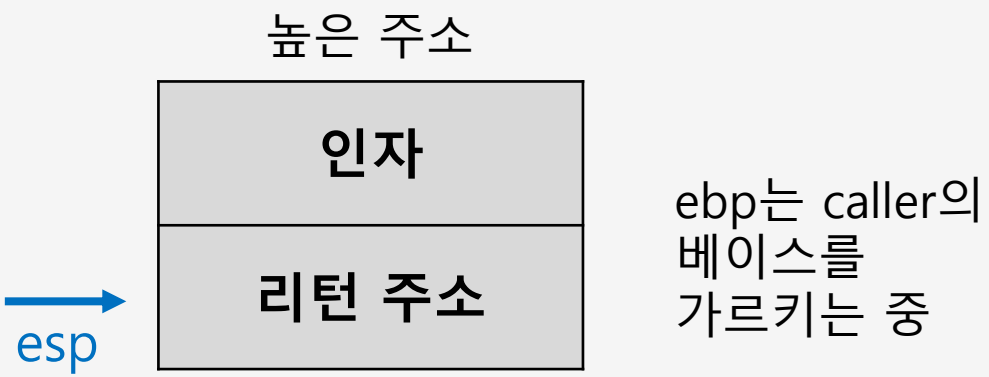
함수 종료 : 에필로그(Epilogue)

callee

```
leave
    : mov esp, ebp
      pop ebp
ret
    : pop eip
      jmp eip
```

caller

```
add esp, 0x4
```



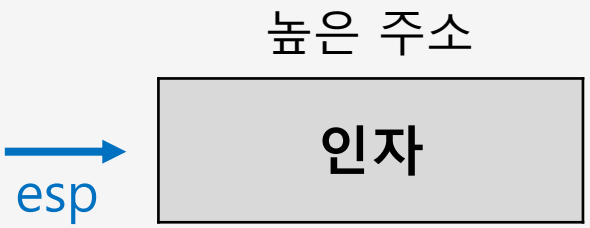
함수 종료 : 에필로그(Epilogue)

callee

```
leave
    : mov esp, ebp
      pop ebp
ret
    : pop eip
      jmp eip
```

caller

```
add esp, 0x4
```



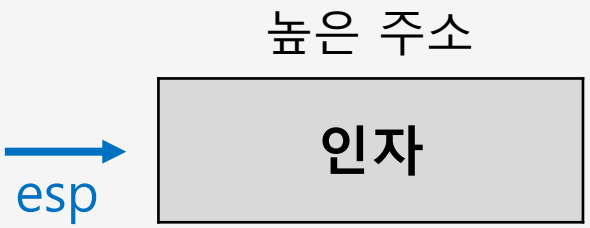
함수 종료 : 에필로그(Epilogue)

callee

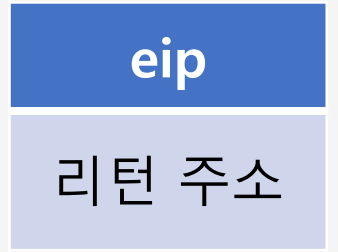
```
leave
    : mov esp, ebp
      pop ebp
ret
    : pop eip
      jmp eip
```

caller

```
add esp, 0x4
```



eip에 저장된 주소로 점프



낮은 주소

함수 종료 : 에필로그(Epilogue)

callee

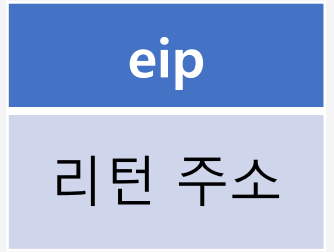
```
leave
    : mov esp, ebp
      pop ebp
ret
    : pop eip
      jmp eip
```

caller

```
add esp, 0x4
```



eip에 저장된 주소로 점프



낮은 주소

프롤로그/에필로그

- 프롤로그 : callee가 자기 스택프레임 만들기

```
push ebp                ; 이전 ebp 백업
mov ebp, esp            ; ebp를 현재 스택프레임 기준점으로
sub esp, 0x10           ; 로컬 변수 공간 확보
```

- 에필로그 : callee가 자기 스택프레임 치우기

```
leave           ; mov esp, ebp + pop ebp
ret             ; pop eip, jmp eip
```

스택을 어떻게 쓰는지 알아보기

```c
#include <stdio.h>

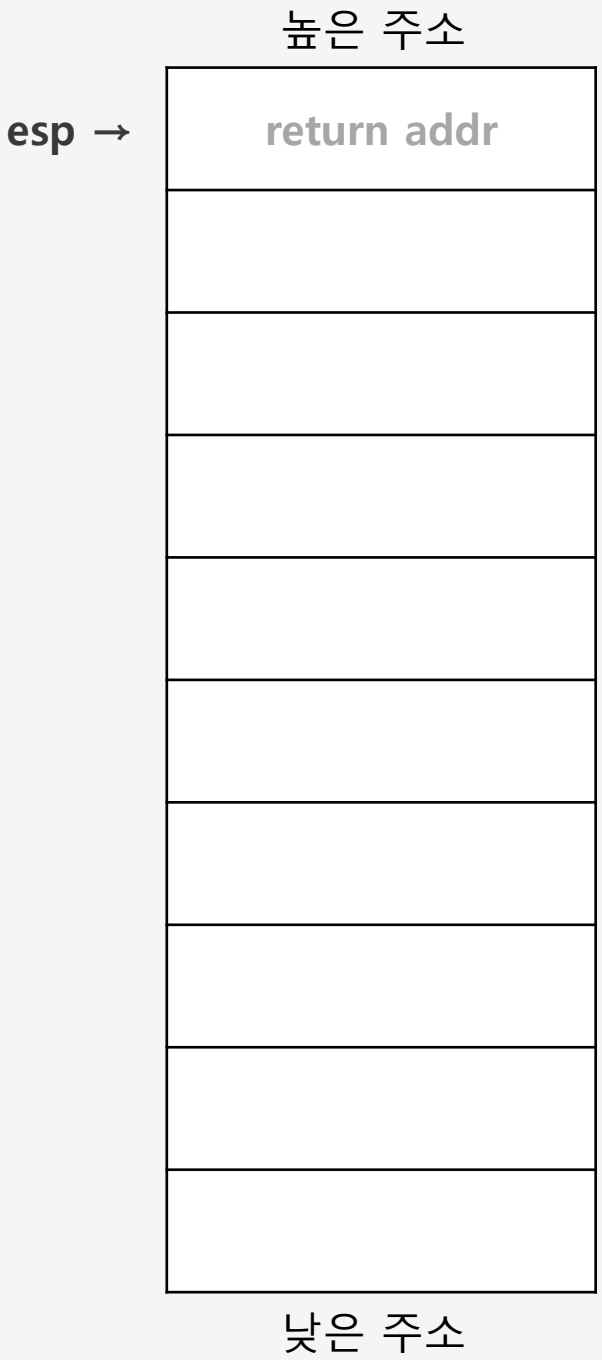
int add(int a, int b) {
    int result;
    result = a + b;
    return result;
}

int main() {
    int x = 3;
    int y = 5;
    int sum;

    sum = add(x, y);
    printf("sum = %d\n", sum);

    return 0;
}
```

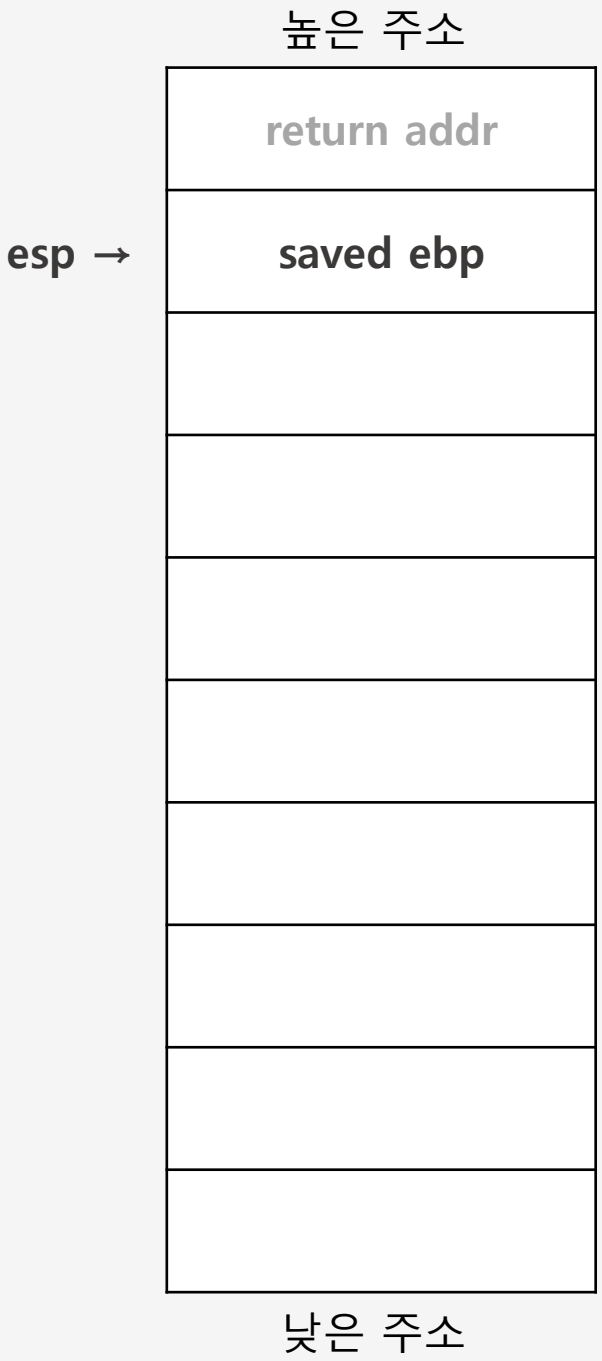
프롤로그
main






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

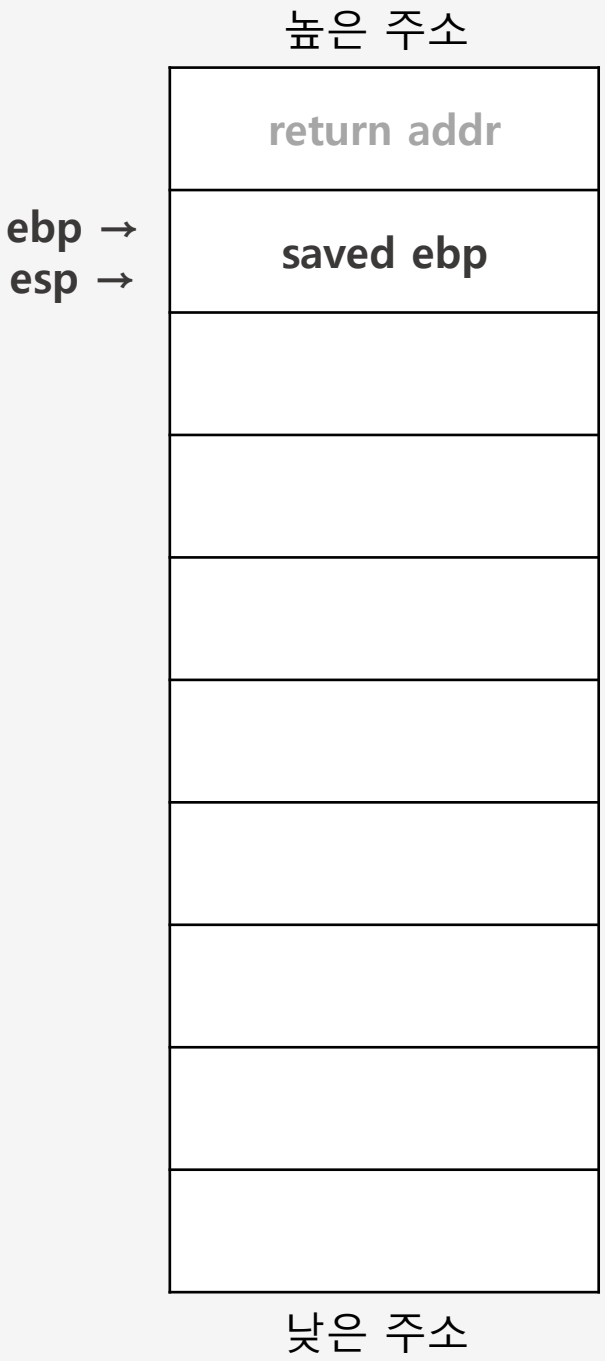
프롤로그
main






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

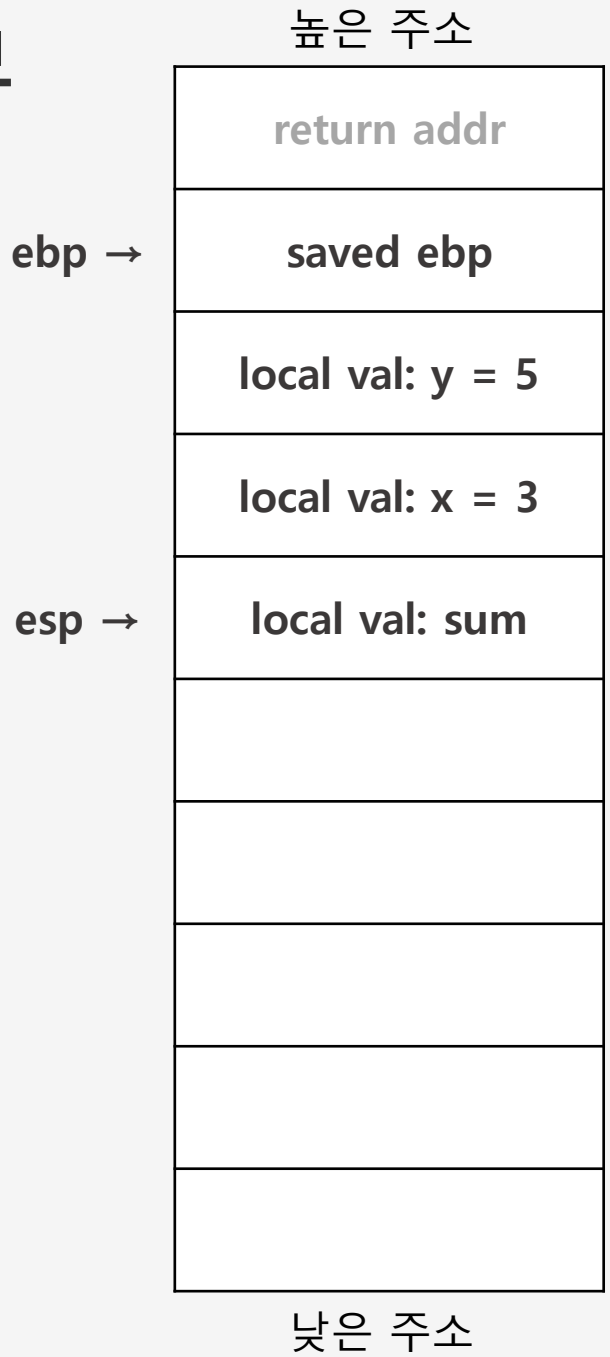
프롤로그
main



eax

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

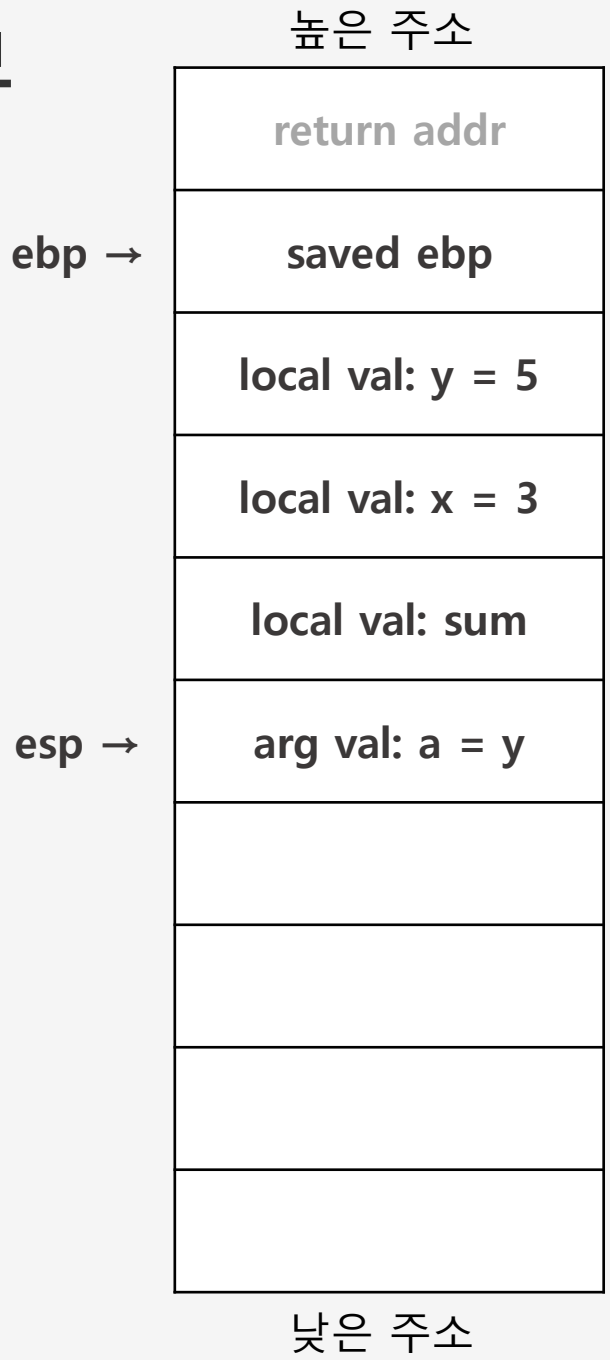
공간 확보
main






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

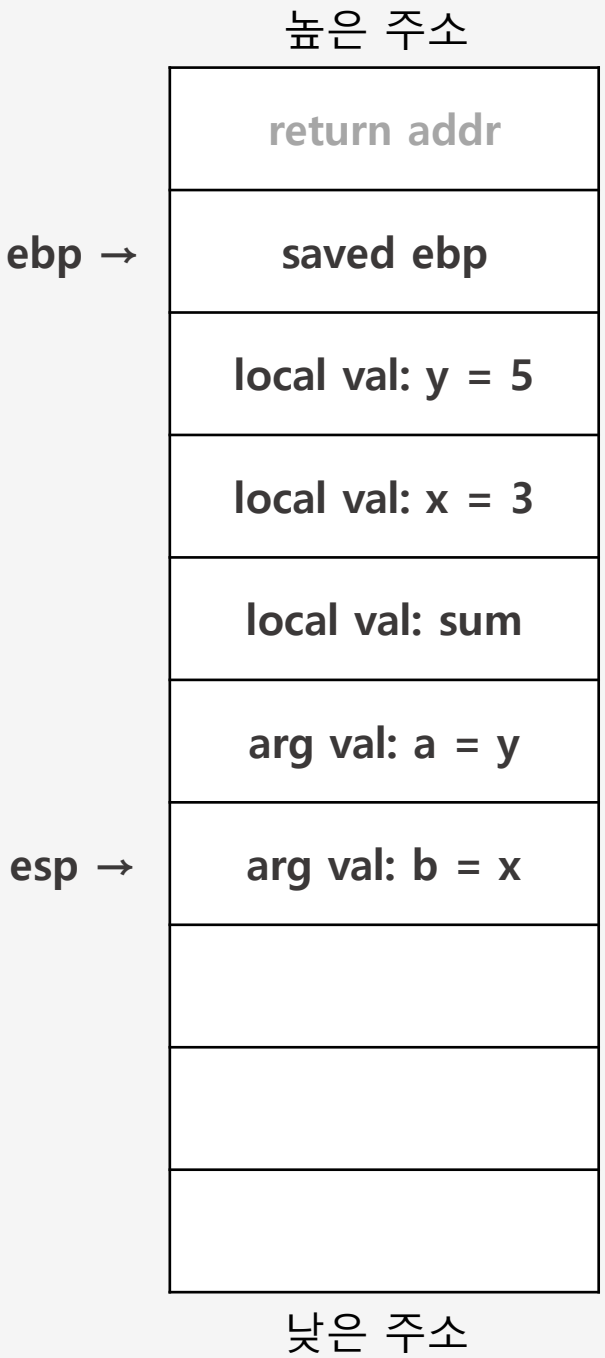
공간 확보
main






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

인자 넣기
main






```c
#include <stdio.h>

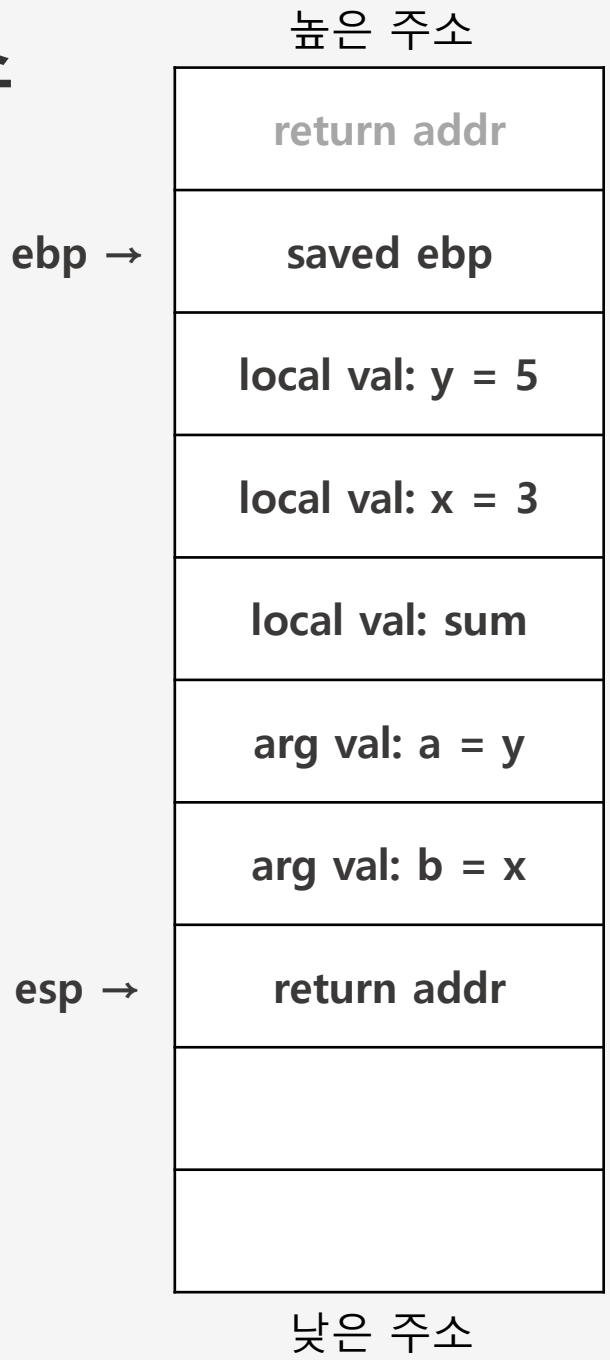
int add(int a, int b) {
    int result;
    result = a + b;
    return result;
}

int main() {
    int x = 3;
    int y = 5;
    int sum;

    sum = add(x, y);
    printf("sum = %d\n", sum);

    return 0;
}
```

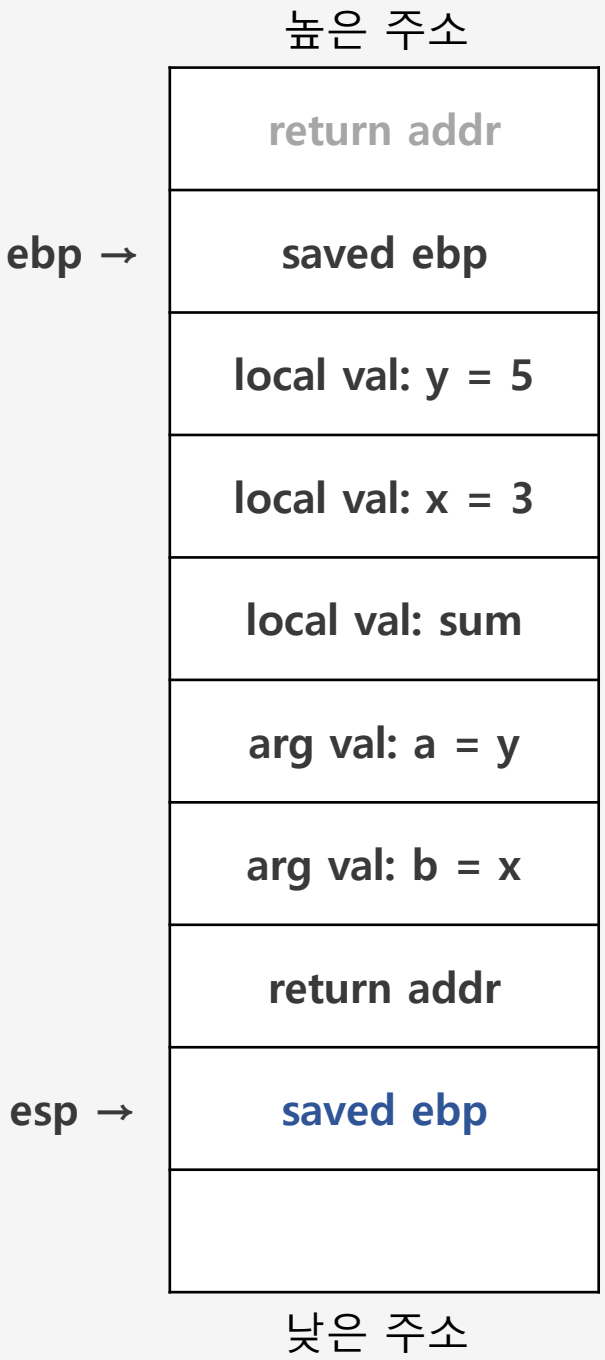
리턴 주소
main






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

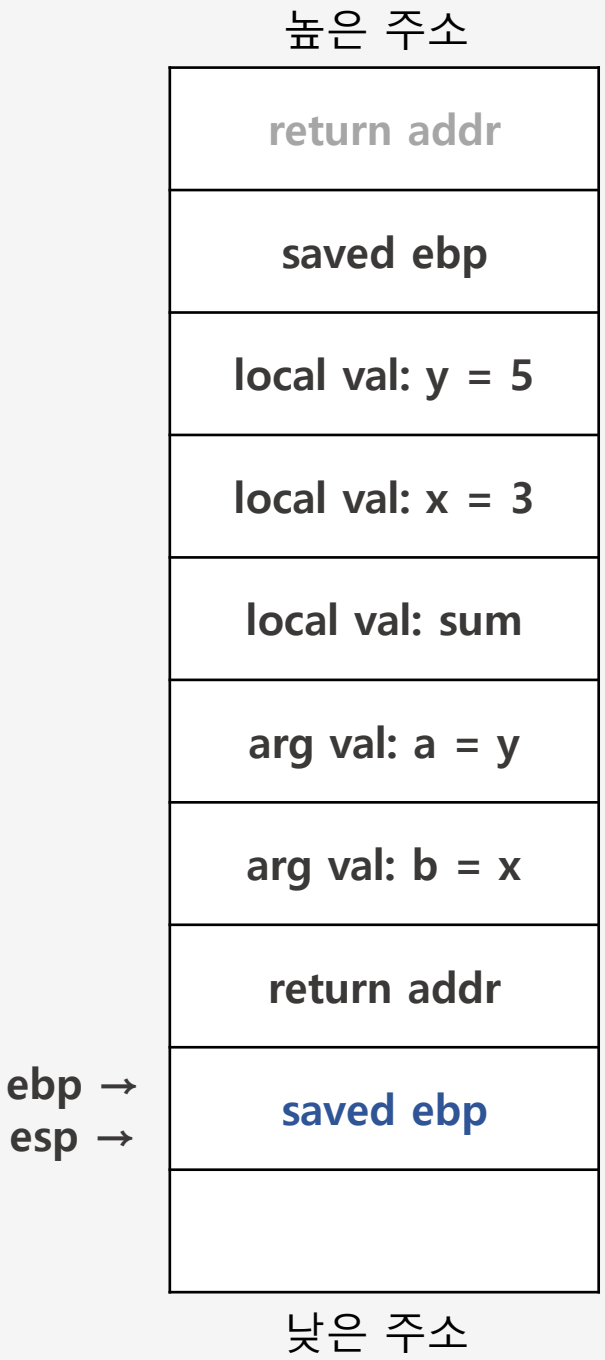
프롤로그
add






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

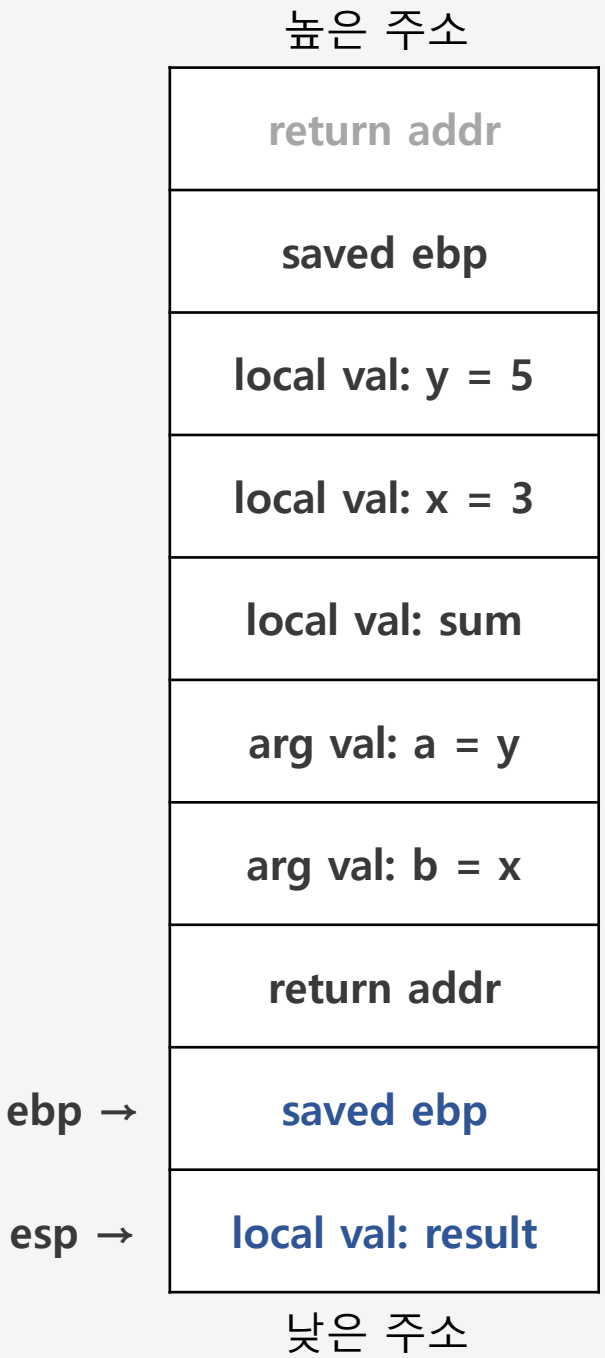
프롤로그
add






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

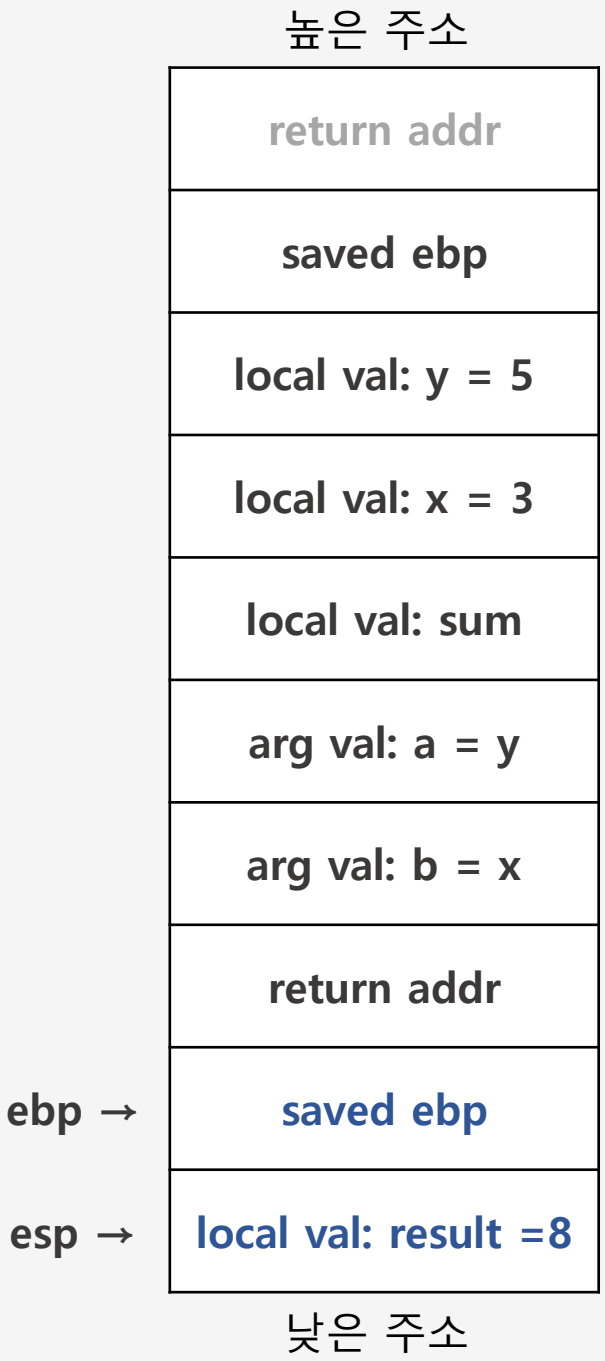
프롤로그
add






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

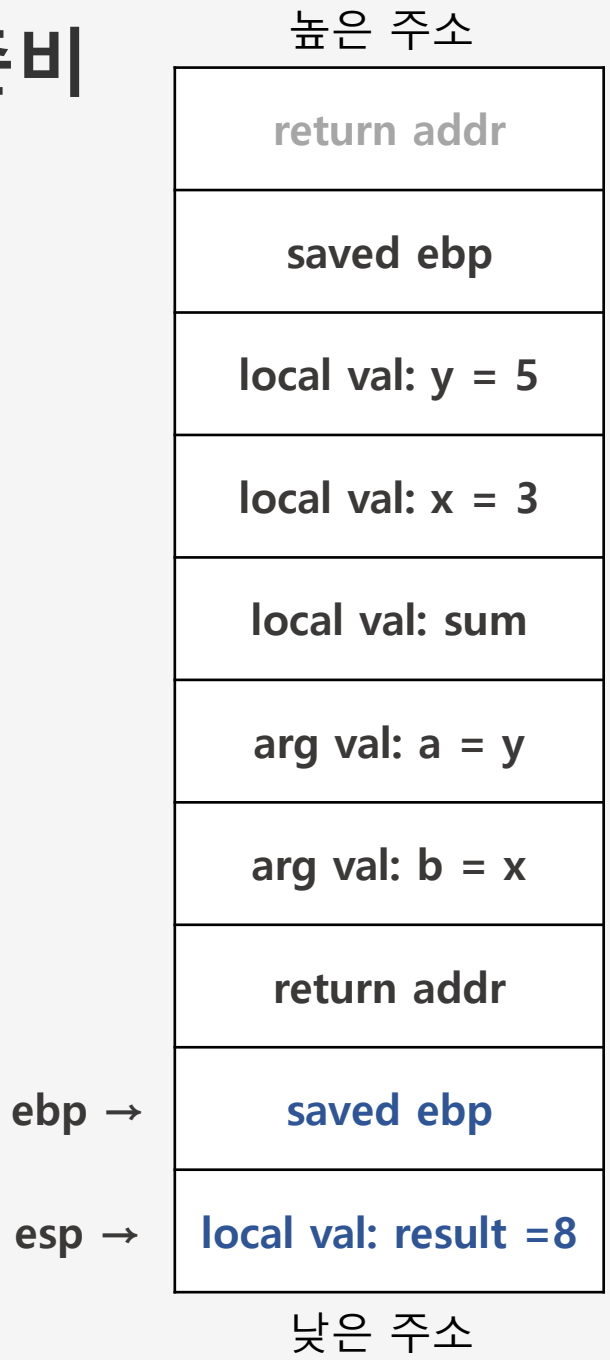
연산
add






```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

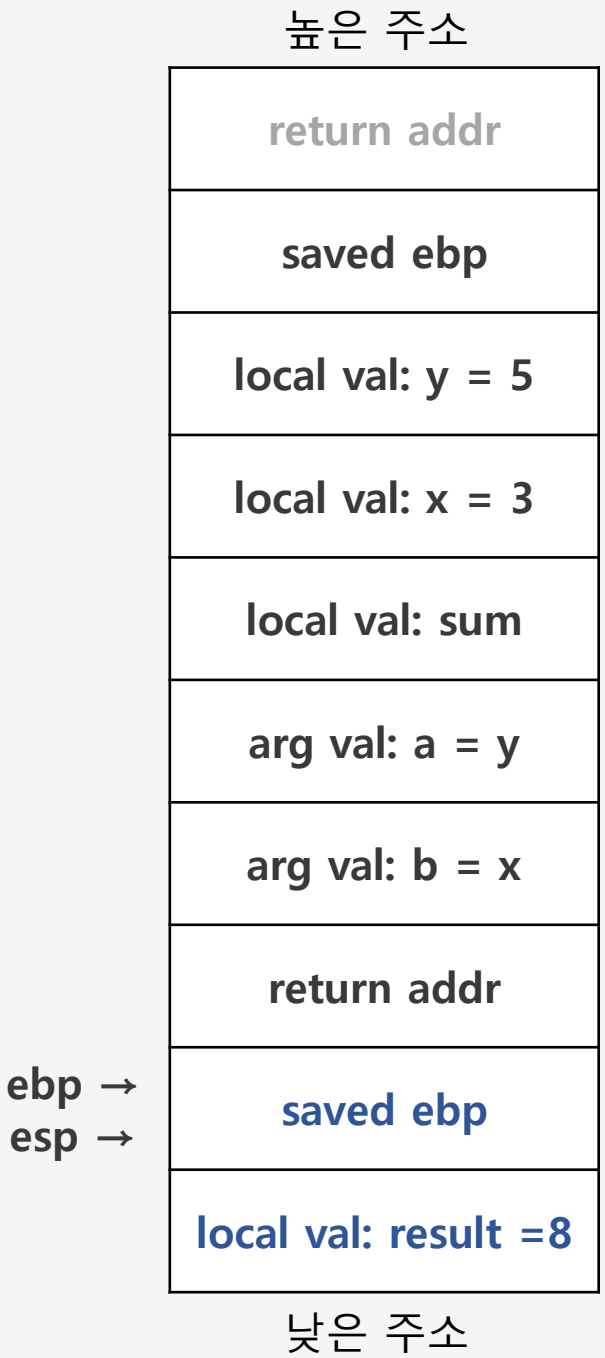
반환값 준비
add



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

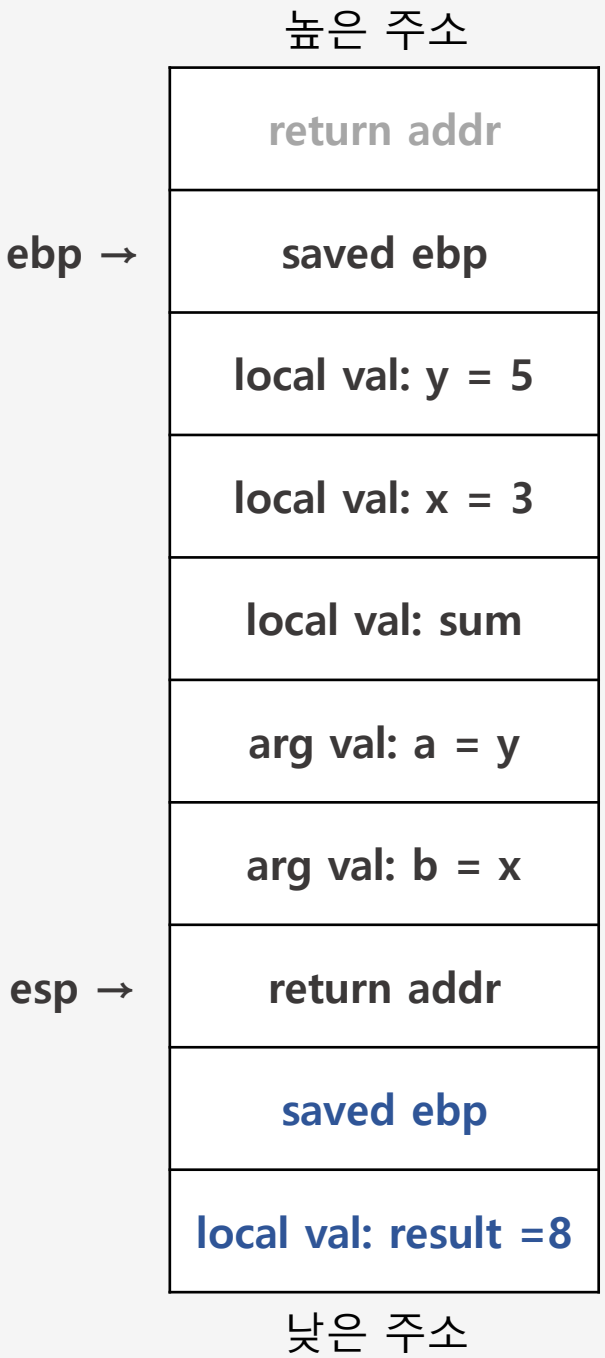
스택 정리
add



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

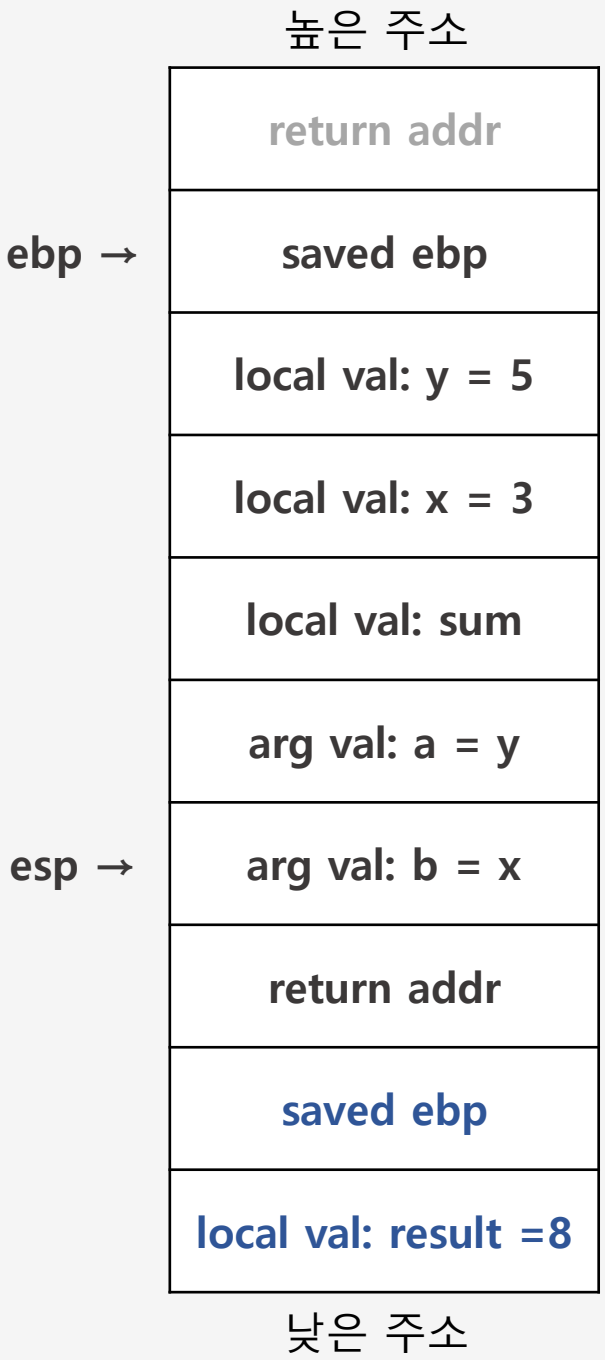
스택 정리
add



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

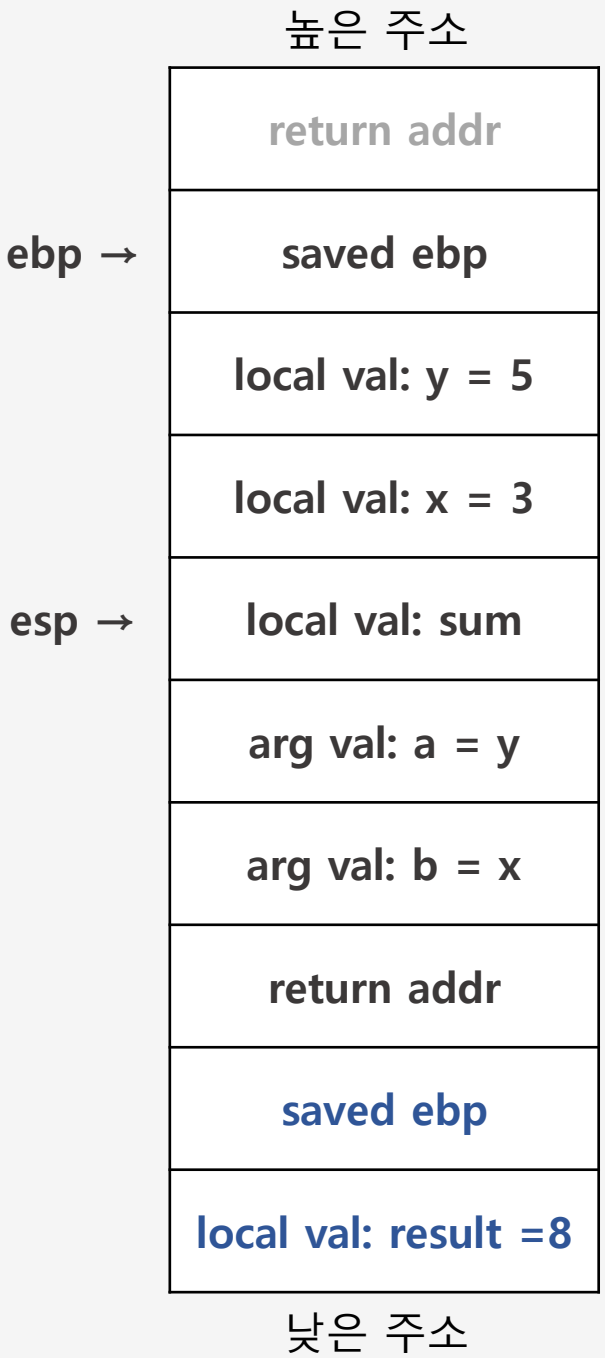
인자 정리
main



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

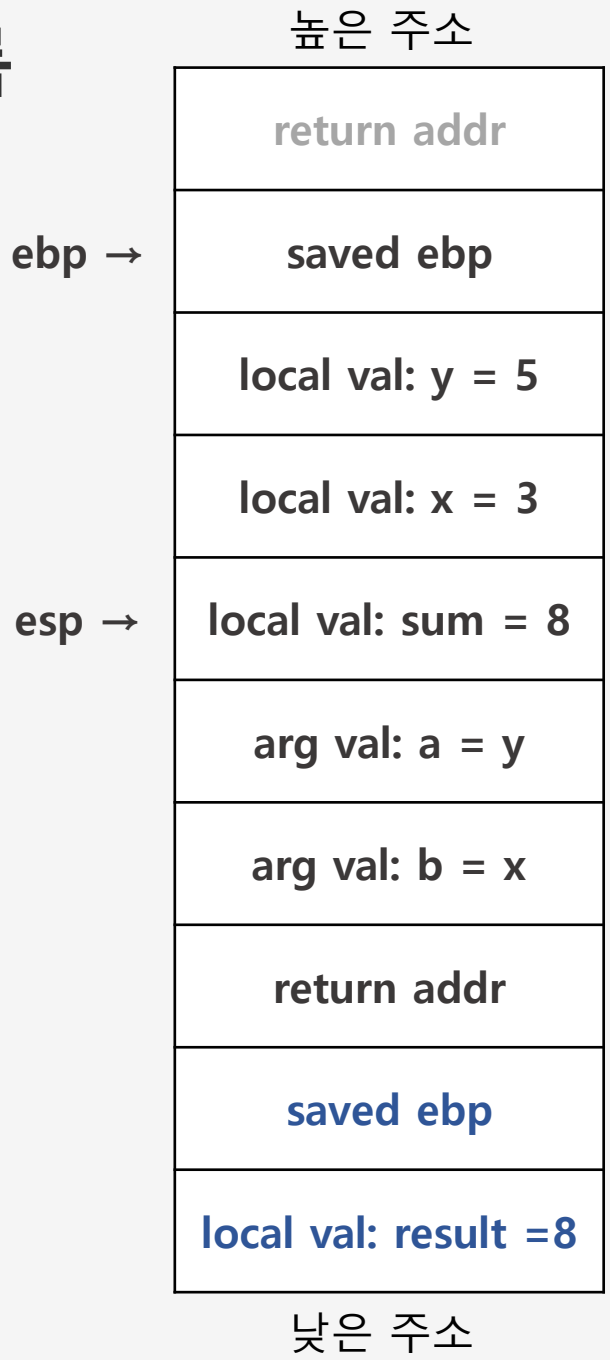
인자 정리
main



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

결과 기록
main



eax
8

```
 1 #include <stdio.h>
 2
 3 int add(int a, int b) {
 4     int result;
 5     result = a + b;
 6     return result;
 7 }
 8
 9 int main() {
10     int x = 3;
11     int y = 5;
12     int sum;
13
14     sum = add(x, y);
15     printf("sum = %d\n", sum);
16
17     return 0;
18 }
```

실습

프로그램이 실행되는 거 구경하기

gcc

- GNU Compiler Collection의 약자
- GNU 프로젝트의 오픈 소스 컴파일러 (CLI 기반)
- C/C++ 등 언어를 컴파일 가능
- 기본 사용법

```
gcc -[옵션] -o [출력파일] [소스파일]
```

gdb

- GNU 프로젝트의 디버거 (CLI 기반)
- C/C++ 등 다양한 언어 지원
- 디버깅에 필요한 여러 기능 제공
- 기본 사용법

```
gdb [프로그램명]
(gdb) [명령어]
```

gdb 명령어

명령어	설명	사용 예시
run / r	프로그램 실행	(gdb) run
break / b [함수명]	해당 함수명 시작점에 브레이크포인트 설정	(gdb) break main
next / n	다음 줄로 이동 (함수 내부 안 들어감)	(gdb) next
step / s	다음 줄 실행 (함수 내부로 들어감)	(gdb) step
continue	다음 브레이크포인트까지 실행	(gdb) continue

gdb 명령어

명령어	설명	사용 예시
print / p [변수명]	변수 값 보기	(gdb) print x
info registers	레지스터 전체 보기	(gdb) info registers
disas / disassemble [함수명]	디스어셈블링, 어셈블리 보기	(gdb) disas main
x/[범위][출력형식] [함수명/주소]	메모리 상태 검사	(gdb) x/20x $esp

같이 해보기

1. 가상머신 환경 세팅: 제가 준비해옴
2. 소스코드 작성: 제가 준비해옴
3. gcc로 소스코드 컴파일
4. gdb로 어셈블리 보기
5. gdb로 메모리 보기

1. 가상머신 환경 세팅

- 자료 구성

```
pwn-intro-prac
 ├──── LICENSE
 ├──── README.md          // 자료 설명
 ├──── setup.sh           // 환경 세팅 스크립트
 ├──── week1              // 1회차 자료
 │     └──── stack.c
 └──── week2              // 2회차 자료
       ├──── bof.c
       ├──── build.sh
       └──── payload.py
```

1. 가상머신 환경 세팅

⚠️ git이 없어요 → 설치하세요

sudo apt update #업데이트
sudo apt install git #설치
git --version #확인

- 실습 자료 받고, 환경 세팅 스크립트 실행

```
# 실습 자료 받기
$ git clone https://github.com/sage-502/pwn-intro-prac

# 실습 디렉터리로 이동
$ cd pwn-intro-prac

# 스크립트 실행
$ sudo bash setup.sh
```

2. 소스코드 작성

- week1 디렉터리에 있는 stack.c확인

```
# 1회차 실습 디렉터리로 이동
$ cd week1

# 실습용 소스코드 확인
$ cat stack.c
```

3. gcc로 소스코드 컴파일

- stack.c 컴파일

```
# 컴파일
$ gcc -m32 -O0 -fno-pie -no-pie ₩
    -mpreferred-stack-boundary=2 ₩
    -fno-stack-protector ₩
    -fno-omit-frame-pointer ₩
    stack.c -o stack
```

```
# 확인
$ ls -l
$ file stack
```

4. gdb로 어셈블리 보기

```
# gdb 실행
$ gdb stack

# 설정
(gdb) set disassembly-flavor intel
(gdb) set pagenation off

# 디스어셈블링
(gdb) disas main
(gdb )disas add
```

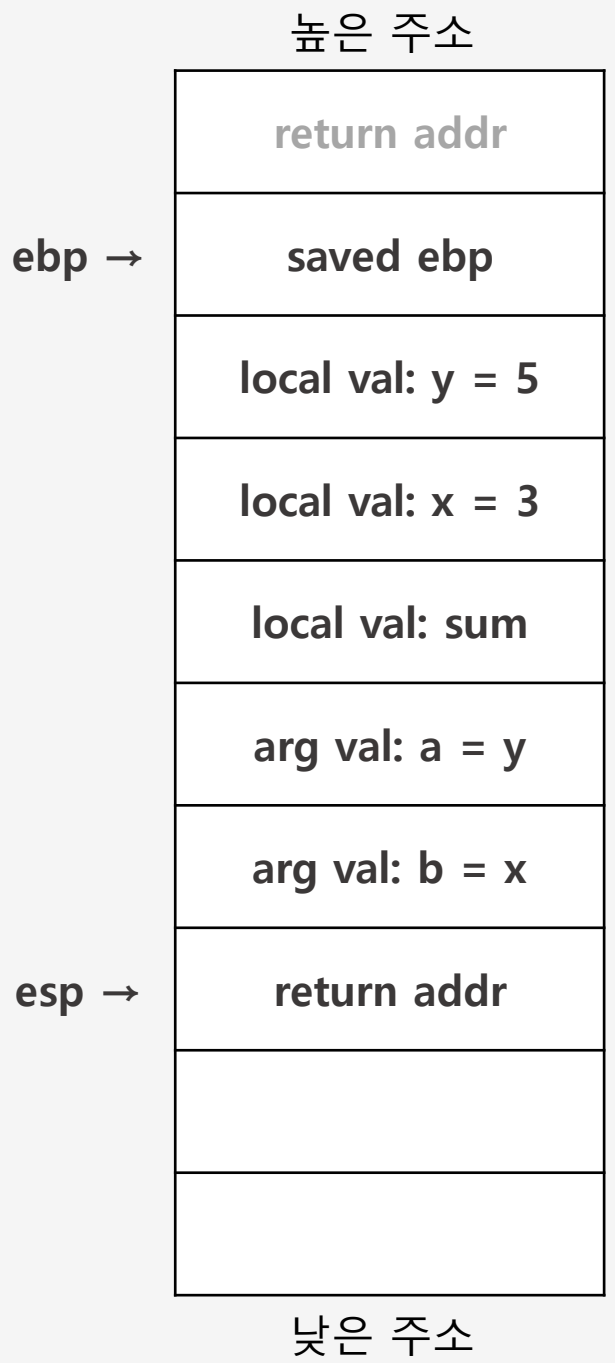
5. gdb로 메모리 보기

```
# 확인하고 싶은 곳에 break 걸기
방법1: (gdb) b <함수명>
방법2: (gdb) b *<명령줄 주소>
방법3: (gdb) b *<함수명 + 명령줄 번호>

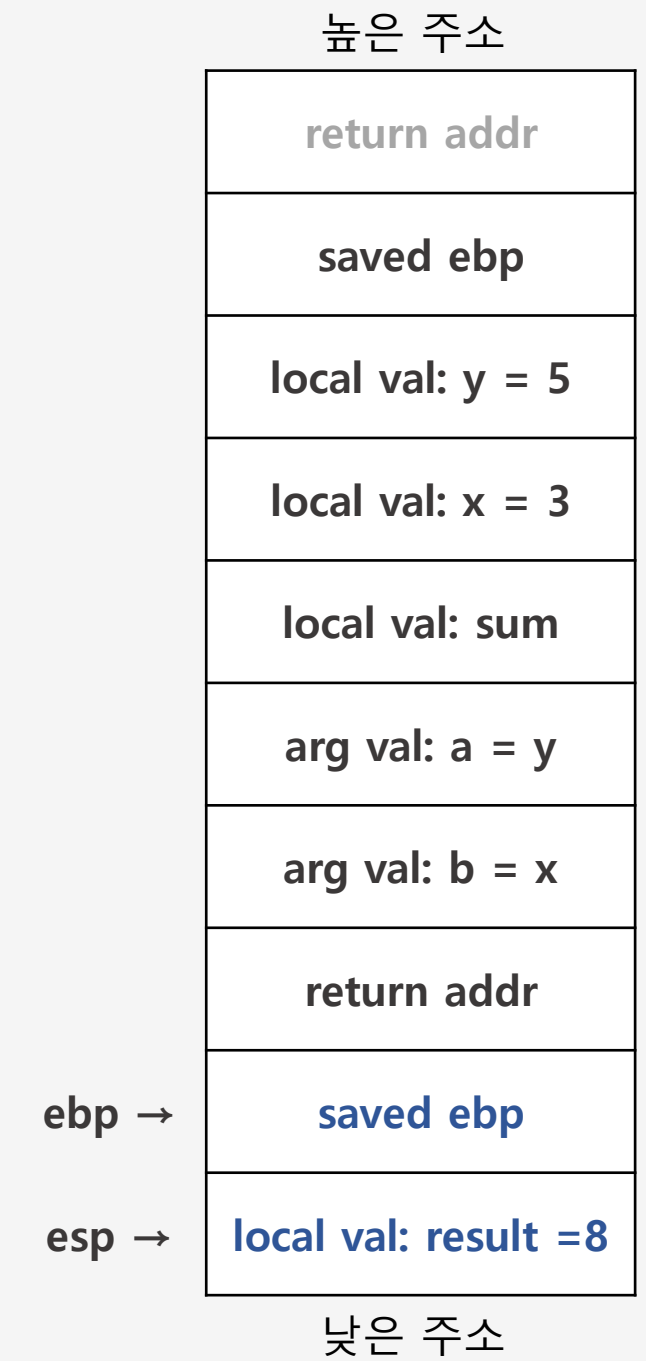
# 브레이크 포인트 확인
(gdb) info b

# 잘못 설정한 거 지우기
(gdb) delete <브레이크 포인트 번호>
```

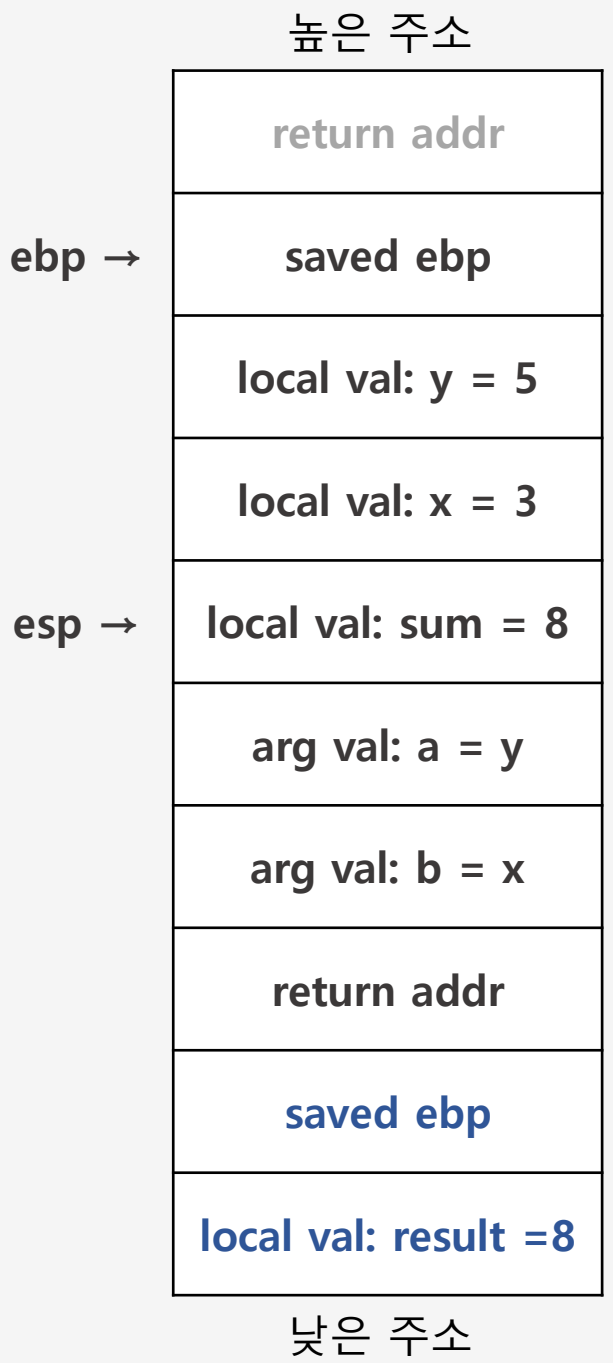
```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack...
(No debugging symbols found in stack)
(gdb) set disassembly-flavor intel
(gdb) set pagination off
(gdb) disas main
Dump of assembler code for function main:
   0x0804918c <+0>:     push   ebp
   0x0804918d <+1>:     mov    ebp,esp
   0x0804918f <+3>:     sub    esp,0xc
   0x08049192 <+6>:     mov    DWORD PTR [ebp-0x4],0x3
   0x08049199 <+13>:    mov    DWORD PTR [ebp-0x8],0x5
   0x080491a0 <+20>:    push   DWORD PTR [ebp-0x8]
   0x080491a3 <+23>:    push   DWORD PTR [ebp-0x4]
   0x080491a6 <+26>:    call   0x8049176 <add>
   0x080491ab <+31>:    add    esp,0x8
   0x080491ae <+34>:    mov    DWORD PTR [ebp-0xc],eax
   0x080491b1 <+37>:    push   DWORD PTR [ebp-0xc]
   0x080491b4 <+40>:    push   0x804a008
   0x080491b9 <+45>:    call   0x8049050 <printf@plt>
   0x080491be <+50>:    add    esp,0x8
   0x080491c1 <+53>:    mov    eax,0x0
   0x080491c6 <+58>:    leave
   0x080491c7 <+59>:    ret
End of assembler dump.
```



```
(gdb) disas add
Dump of assembler code for function add:
   0x08049176 <+0>:     push   ebp
   0x08049177 <+1>:     mov    ebp,esp
   0x08049179 <+3>:     sub    esp,0x4
   0x0804917c <+6>:     mov    edx,DWORD PTR [ebp±0x8]
   0x0804917f <+9>:     mov    eax,DWORD PTR [ebp±0xc]
   0x08049182 <+12>:    add    eax,edx
   0x08049184 <+14>:    mov    DWORD PTR [ebp-0x4],eax
   0x08049187 <+17>:    mov    eax,DWORD PTR [ebp-0x4]
   0x0804918a <+20>:    leave
   0x0804918b <+21>:    ret
End of assembler dump.
```



```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack...
(No debugging symbols found in stack)
(gdb) set disassembly-flavor intel
(gdb) set pagination off
(gdb) disas main
Dump of assembler code for function main:
   0x0804918c <+0>:     push   ebp
   0x0804918d <+1>:     mov    ebp,esp
   0x0804918f <+3>:     sub    esp,0xc
   0x08049192 <+6>:     mov    DWORD PTR [ebp-0x4],0x3
   0x08049199 <+13>:    mov    DWORD PTR [ebp-0x8],0x5
   0x080491a0 <+20>:    push   DWORD PTR [ebp-0x8]
   0x080491a3 <+23>:    push   DWORD PTR [ebp-0x4]
   0x080491a6 <+26>:    call   0x8049176 <add>
   0x080491ab <+31>:    add    esp,0x8
   0x080491ae <+34>:    mov    DWORD PTR [ebp-0xc],eax
   0x080491b1 <+37>:    push   DWORD PTR [ebp-0xc]
   0x080491b4 <+40>:    push   0x804a008
   0x080491b9 <+45>:    call   0x8049050 <printf@plt>
   0x080491be <+50>:    add    esp,0x8
   0x080491c1 <+53>:    mov    eax,0x0
   0x080491c6 <+58>:    leave
   0x080491c7 <+59>:    ret
End of assembler dump.
```



과제 소개

복습용 과제

과제: 강의 내용 정리해오기

- 이론 파트 + 실습 파트 정리
- 이론 파트: 본인이 이해한대로 정리하기
- 실습 파트: 실습 화면 스크린샷과 직접 그린 스택 프레임 이미지 첨부

제출 방식

- 블로그, 노션, 깃허브 등 자유.
- 공개글로 작성한 후 동아리 노션에 링크를 올려주세요.
- 기한 : 다음주 동아리 시작 전까지 (11/5 18:30까지)
    - 지각제출 : 금요일까지 (11/6 23:59)

요약

- 포너블 개요
- 메모리 4영역
  - 코드, 데이터, 힙, 스택
- 레지스터
  - eip, esp, ebp, eax
- 스택프레임
  - 함수 호출 규약, 프롤로그, 에필로그

2주차 스포일러

- BOF
- ret2win
- 쉘 획득 실습